

Introduction

Palindrome problems show up constantly in coding interviews because they test something deeper than string manipulation — they test whether you can spot structure. Most candidates see "palindrome" and think "two pointers, check if it reads the same backwards." That instinct works for validation problems. It falls apart the moment you're asked to *construct* something, especially the k-th smallest arrangement out of potentially millions of valid ones.

Smallest Palindromic Rearrangement II is a great example of a problem that looks intimidating — it's rated Hard on LeetCode — but becomes very manageable once you notice one thing: a palindrome is defined entirely by its first half. Interviewers use this problem to see if you reach for brute force out of habit, or if you look for the symmetry that cuts the problem size roughly in half (and, more importantly, avoids ever generating the full search space).

Learning this pattern pays off beyond this one problem. Ranking arrangements without listing them — using counting instead of generation — is a recurring interview technique that appears in problems about k-th permutations, lexicographic ordering, and combinatorial enumeration in general.

Problem Overview

You're given a string `s` that is already a palindrome, and an integer `k`. Your task is to find the k-th smallest string, in alphabetical order, that you can form by rearranging the letters of `s`, such that the result is *also* a palindrome.

If there aren't at least `k` distinct palindromic rearrangements possible, return an empty string.

A few things worth internalizing before you touch any code:

- The input is guaranteed to already be a palindrome, so you never have to worry about more than one letter having an odd count — that edge case is handled for you by the problem's constraints.
- "Rearrangement" here specifically means: same multiset of letters, different order, and the result must still be a palindrome.
- Rankings are alphabetical, treating the full string as a word you'd find in a dictionary.

Example

Take `s = "cbabc"` and `k = 2`.

The letters are: two `b`'s... wait, let's count precisely — `c`, `b`, `a`, `b`, `c`. That's `a : 1`, `b : 2`, `c : 2`.

Since only `a` has an odd count, `a` sits in the middle. The remaining letters split into a left half and a mirrored right half. The left half must be built from one `b` and one `c`.

The possible left halves, sorted alphabetically, are: 1. `"bc"` 2. `"cb"`

Mirroring each around the middle `a` gives the two full palindromes: 1. `"bcacb"` → wait — mirroring `"bc"` gives `"bc" + "a" + "cb" = "bcacb"` 2. `"cb" + "a" + "bc" = "cbabc"`

Since `k = 2`, the answer is `"cbabc"` — which happens to be the original string. This matches the transcript's walkthrough: ranking only ever requires reasoning about the tiny left half, not the full string.

Intuition

Here's how an experienced engineer approaches this rather than jumping straight to code.

Step one: exploit the symmetry. A palindrome's right half is completely determined by its left half — it's just the left half reversed. That means "rearrange this string into a palindrome" is really "rearrange *half* the letters into any order, then mirror it." If a letter count is odd, exactly one letter is left over and it goes in the dead center. The problem's search space just shrank by roughly half.

Step two: recognize this is a ranking problem, not a generation problem. You don't need to know *what* the first, second, or hundredth arrangement looks like in order to find the `k`-th one — you need to know *how many* arrangements start with each candidate letter. This is the same idea behind converting a number to a factorial-based positional system: at each position, you ask "how many things could come after this choice?" and use that count to skip entire blocks of possibilities at once.

Step three: pick greedily, one letter at a time. Sort the distinct letters in the left half alphabetically. For the smallest letter, ask: "if I place this letter first, how many valid palindromes can follow?" If `k` fits within that number, commit to the letter and recurse into the next position with the same `k`. If not, subtract that count from `k` and move to the next letter. This is exactly like skipping past whole sections of a phone book instead of reading every name.

Step four: mirror and finish. Once the left half is fully decided, stitch it together, insert the middle letter if there is one, and append the reverse of the left half. No further decisions needed.

Brute Force Solution

Idea: Generate every permutation of the string, filter down to only the ones that are palindromes, deduplicate, sort alphabetically, and index into position `k - 1`.

Algorithm: 1. Generate all permutations of `s` (e.g. with `itertools.permutations`). 2. Filter for strings equal to their own reverse. 3. Put results in a set to remove duplicates. 4. Sort the set. 5. Return the `k`-th element, or empty string if there are fewer than `k`.

Advantages: Trivial to write, trivial to trust, works fine for tiny strings.

Disadvantages: The number of permutations of a string grows factorially with length. Even a modest string produces a rearrangement count in the millions, and most of those get thrown away for not being palindromes — wasted work at massive scale.

Time complexity: $O(n! \cdot n)$ — generating $n!$ permutations, each requiring $O(n)$ work to build and check. **Space complexity:** $O(n! \cdot n)$ to store the permutations before filtering.

Optimal Solution

The optimal approach never generates a full string until it has already made every decision. It works entirely on counts.

Step 1 — Build the frequency table and find the middle letter. Count each letter with a hash map. If any letter has an odd count, it becomes the fixed middle character; every letter's count (after removing one for the middle letter, if applicable) is halved to get the multiset for the left half.

Step 2 — Count total arrangements using the multinomial formula. For a multiset with total length `L` and letter counts `c1, c2, ..., cm`, the number of distinct arrangements is:

$$L! / (c1! \cdot c2! \cdot \dots \cdot cm!)$$

If this total is less than `k`, return `""` immediately — no further work needed.

Step 3 — Greedily choose each letter of the left half. Walk positions left to right. At each position, try letters in alphabetical order. For a candidate letter, temporarily use one occurrence of it and compute how many arrangements are possible for the *remaining* letters at the *remaining* positions (same multinomial formula, smaller counts). Call this `count`.

Condition

Action

`k <= count` Lock in this letter, move to next position, keep same `k`

Condition**Action**

`k > count` Subtract `count` from `k`, try the next letter

Step 4 — Mirror. Concatenate: `left_half + middle_letter (if any) + reversed(left_half)`.

This walks past huge blocks of the search space in single arithmetic operations instead of enumerating them.

Python Code

```
from collections import Counter
from math import factorial

def kth_palindrome_rearrangement(s: str, k: int) -> str:
    counts = Counter(s)

    middle = ""
    half_counts = {}
    for letter, count in counts.items():
        if count % 2 == 1:
            middle = letter
            count -= 1
        if count:
            half_counts[letter] = count // 2

    half_length = sum(half_counts.values())

    def arrangements(remaining_counts, remaining_length):
        total = factorial(remaining_length)
        for count in remaining_counts.values():
            total //= factorial(count)
        return total

    total_arrangements = arrangements(half_counts, half_length)
    if k > total_arrangements:
        return ""

    left_half = []
    remaining = dict(half_counts)
    remaining_length = half_length

    for _ in range(half_length):
        for letter in sorted(remaining):
            if remaining[letter] == 0:
                continue

            remaining[letter] -= 1
```

```

remaining_length -= 1
count = arrangements(remaining, remaining_length)

if k <= count:
    left_half.append(letter)
    if remaining[letter] == 0:
        del remaining[letter]
    break
else:
    k -= count
    remaining[letter] += 1
    remaining_length += 1

left_str = "".join(left_half)
return left_str + middle + left_str[::-1]

```

Code Walkthrough

- `Counter(s)` builds the frequency table in one pass — no manual dictionary bookkeeping needed.
- The loop over `counts.items()` finds the odd-count letter (there's guaranteed to be at most one, since `s` is a valid palindrome) and builds `half_counts`, the letter counts for just the left half.
- `arrangements()` implements the multinomial formula directly: factorial of the total length divided by the factorial of each letter's count. It's called both once up front (to validate `k`) and repeatedly during greedy selection (to test each candidate letter).
- The greedy loop tries letters in sorted order at each position. It provisionally "spends" one occurrence of the letter, checks how many arrangements the rest of the positions allow, and either commits (`k <= count`) or backs out and subtracts `count` from `k` before trying the next letter.
- The final line builds the palindrome from the completed left half, the middle character (empty string if none), and the reversed left half.

Dry Run

```
s = "cbabc", k = 2.
```

```
counts = {c: 2, b: 2, a: 1}. a is odd → middle = "a". half_counts = {c: 1, b: 1},
half_length = 2.
```

`total_arrangements = arrangements({c:1, b:1}, 2) = 2! / (1! · 1!) = 2`. Since `k = 2 ≤ 2`, continue.

Position 0: Try `b` first (alphabetical). Spend one `b` → `remaining = {c:1, b:0}` → cleaned to `{c:1}`, `remaining_length = 1`. `arrangements({c:1}, 1) = 1`. Is `k(2) ≤ 1`? No. So `k -= 1` → `k = 1`. Restore `b`. Try `c`: spend one `c` → `remaining = {b:1}`, `remaining_length = 1`. `arrangements = 1`. Is `k(1) ≤ 1`? Yes. Commit `c`. `left_half = ["c"]`.

Position 1: Only `b` remains. Spend it → `remaining = {}`, `remaining_length = 0`. `arrangements = 1`. `k(1) ≤ 1`? Yes. Commit `b`. `left_half = ["c", "b"]`.

Result: `"cb" + "a" + "bc" = "cbabc"` — matches expected output.

Complexity Analysis

Time: $O(n^2 \cdot \log(\text{max_count}))$ roughly — for each of the $n/2$ positions, we try up to 26 letters, and each `arrangements()` call does $O(n)$ work computing factorials (or $O(1)$ if factorials are precomputed). In practice this is dominated by $O(n \cdot 26) \approx O(n)$ per position, giving $O(n^2)$ overall for a small alphabet — dramatically better than the brute force's factorial blowup.

Space: $O(n)$ for the counters and the result string.

Why correct: The multinomial formula exactly counts distinct arrangements of a multiset (it divides out arrangements that would otherwise be counted as different but produce identical strings due to repeated letters). The greedy step is correct because trying letters in sorted order and comparing `k` against the count of arrangements starting with each letter is precisely how lexicographic ranking works — it's the same logic used to find the k -th permutation of a set, adapted for multisets with repeated letters.

Alternative Solutions

A middle-ground approach: generate arrangements lazily with a recursive generator that yields palindromic rearrangements in sorted order, and stop at the k -th one without generating the rest. This avoids storing all arrangements in memory (unlike full brute force) but still does $O(k)$ work in the worst case, which can be just as bad as brute force if `k` is large. The counting-based approach is strictly better because it computes the answer in polynomial time regardless of how large `k` is, as long as it's within the valid range.

Edge Cases

- **All letters identical** (e.g. "aaaa"): only one arrangement exists. `k` must be exactly 1, otherwise return "".
- **k exceeds total arrangements**: caught immediately after computing `total_arrangements`, before any greedy work begins.
- **Single-character string**: `half_counts` is empty, `middle` is the one character, and the result is just that character — `k` must be 1.
- **String with no repeated letters** aside from the guaranteed odd-count letter: every arrangement is distinct, so `total_arrangements` equals the straightforward factorial of the half-length.
- **k = 1**: should always return the alphabetically smallest valid palindrome without needing any subtraction in the greedy loop.

Common Mistakes

1. **Forgetting to restore state when a candidate letter doesn't fit.** If you spend a letter to test `arrangements()` but the count doesn't cover `k`, you must put it back before trying the next letter.
2. **Recomputing factorials incorrectly for repeated letters.** Forgetting to divide by each letter's factorial count turns the multinomial formula into a plain factorial, wildly overcounting arrangements.
3. **Applying brute force and hitting timeouts** on longer strings — the constraint sizes for this problem are specifically chosen to punish $O(n!)$ approaches.
4. **Mishandling the middle character** when the string has even total length (no odd-count letter) — `middle` should stay an empty string, not `None` or a placeholder that leaks into the final concatenation.
5. **Off-by-one on k**. Since `k` is 1-indexed in the problem statement, comparing `k <= count` (not `k < count`) is what keeps the indexing correct throughout the greedy loop.

Interview Questions

- How would you modify this to return the *k*-th *largest* palindrome instead of smallest?
- Can you compute the rank of a given palindrome instead of finding the palindrome at a given rank?
- What changes if the input string is not guaranteed to be a palindrome, but you're asked whether it *can* be rearranged into one?

- How would you handle extremely large `k` values that exceed standard integer ranges in other languages?
- Can you precompute factorials to speed up repeated `arrangements()` calls?

Similar Problems

- **Permutation Sequence (LeetCode 60):** the direct ancestor of this problem's ranking technique, without the palindrome constraint.
- **Next Permutation (LeetCode 31):** builds intuition for lexicographic ordering of rearrangements.
- **Palindrome Permutation (LeetCode 266):** the yes/no version — can a string be rearranged into a palindrome at all.
- **Palindrome Permutation II (LeetCode 267):** generates all distinct palindromic permutations, useful for understanding why brute force doesn't scale.

Key Takeaways

Palindromes are defined by their left half — every problem involving palindrome construction should start by asking whether you can solve just the half and mirror the rest. Counting arrangements with factorials lets you validate feasibility and skip large chunks of the search space instantly, instead of generating and filtering. Greedy, position-by-position selection combined with a counting formula is the standard technique for any "find the k-th arrangement" problem, palindromic or not.

Watch the Video

For the full step-by-step walkthrough with visuals, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series that takes LeetCode's daily challenge and works through it from first principles — building intuition before writing a single line of code, and always explaining *why* an approach works, not just *what* it does.

Call To Action

If this helped the problem click, give the video a like on YouTube and subscribe for daily walkthroughs. For deeper dives and written breakdowns like this one, subscribe to the Substack newsletter. Drop a comment with your own approach or questions, and share this with anyone prepping for interviews.

Quick Review

What's the pattern name and trigger here?

Half-and-Mirror counting: when a string is a palindrome rearrangement, split counts in half and count arrangements of that half.

Time complexity of counting half-permutations?

$O(n \log n)$ — dominated by sorting/grouping the half-multiset of characters.

Space complexity of this approach?

$O(n)$ or $O(26)$ for character frequency counts of the half string, not the whole string.

Trickiest edge case to watch for?

One odd-count character (the middle letter) breaks the even split — must be pulled out before halving.

Why avoid listing arrangements directly?

Listing is exponential; counting via factorial/multinomial formula on half-counts avoids enumerating candidates.

Follow-up: how do multiset permutation counts work?

$n!$ divided by the product of factorials of each repeated character's count, applied to the half-string only.